Distributed Leaderboard System

1. Workflow

"Our project is a **Distributed Leaderboard System** where multiple users can update scores at the same time and see real-time rankings."

Step-by-step flow:

1. **Client connects to server using TLS (secure connection)**
   → Data is encrypted (secure communication)
2. **User registers / logs in**
   → Managed by UserManager (stores data in database)
3. **User submits score**
   → Request goes to server → Leaderboard engine updates score
4. **Server processes request using threads**
   → Multiple clients handled at same time (concurrent users)
5. **Leaderboard is updated and cached**
   → Sorted ranking is maintained
6. **Client can view leaderboard or their rank instantly**

2. Architecture

"Architecture has 3 main parts"

1. Client

- Sends requests (login, score update, view leaderboard)

2. Server (Main Brain)

- Handles multiple users using **ThreadPool (multi-threading)**

- Each client → one thread

3. Backend Components

- **UserManager** → handles users & database
- **LeaderboardEngine** → handles ranking logic

3. Key Concepts + Logic Used

1. Real-Time Ranking + Caching Logic

"We maintain real-time leaderboard using a dirty flag caching system."

Logic:

- When score updates → mark cache as dirty
- Next read → rebuild leaderboard
- So we don't sort every time → faster performance

From code: cache rebuild only when needed

2. Concurrent Updates + Multi-threading

"We allow multiple clients to update scores at the same time using multi-threading."

- **Logic**:
  - Each client → separate thread
  - ThreadPool handles many users

From server: ThreadPoolExecutor

3. Consistency + Locking Mechanism

"We ensure data consistency using locks."

- **Logic**:
  - Only one thread updates shared data at a time
  - Prevents race conditions

Uses threading.Lock

4. Conflict Resolution + Max Score Logic

"If multiple updates happen, we use max-score logic."

- **Logic**:
  - Store: MAX(old_score, new_score)
  - So lower score never overwrites higher

Implemented in database query

5. High Performance + Efficient Design

"We ensure high performance using multiple optimizations."

- **Logic**:
  - Caching (dirty flag)
  - WAL mode (no read/write blocking)
  - Per-thread DB connections

Supports 500 concurrent users

5. Strengths of Your Project

1. High Performance
   - Handles 500+ users simultaneously
2. Real-time Updates
   - Leaderboard updates instantly
3. Strong Consistency
   - No wrong rankings or stale data
4. Conflict Safe
   - No data loss (max score logic)
5. Secure Communication
   - Uses TLS encryption
6. Scalable Design
   - Can extend to bigger systems

6. Future Improvements

- Use **multiple servers** for better scalability
- Upgrade to **better database (PostgreSQL)**
- Use **advanced caching (Redis)**
- Improve **concurrency model (async instead of threads)**
- Add **better security (proper certificates)**